

## Tutorial 7: Observer Pattern and Event Handling

The Observer design pattern is useful for designing flexible and reusable object-oriented software.

**The Observer pattern addresses the following problems:**

- A one-to-many dependency between objects should be defined without making the objects tightly coupled.
- It should be ensured that when one object changes state an open-ended number of dependent objects are updated automatically.
- It should be possible that one object can notify an open-ended number of other objects.

**What solution does the Observer design pattern describe?**

- Define Subject and Observer objects.
- so that when a subject changes state, all registered observers are notified and updated automatically (and probably asynchronously).

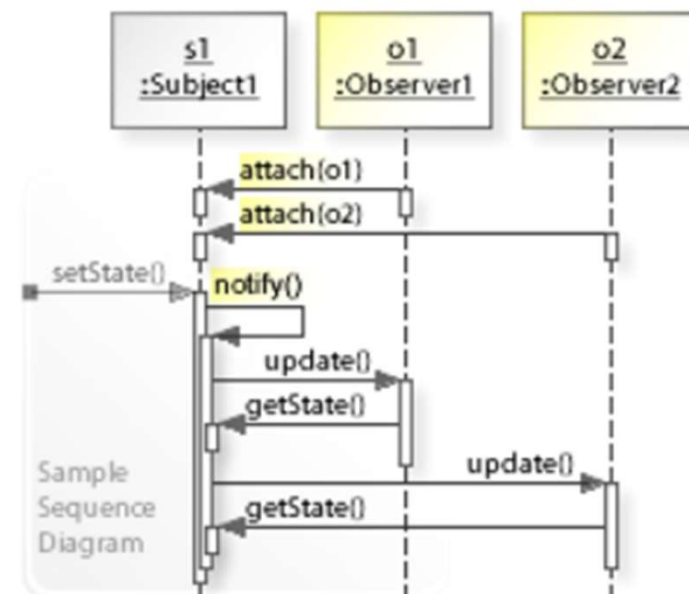
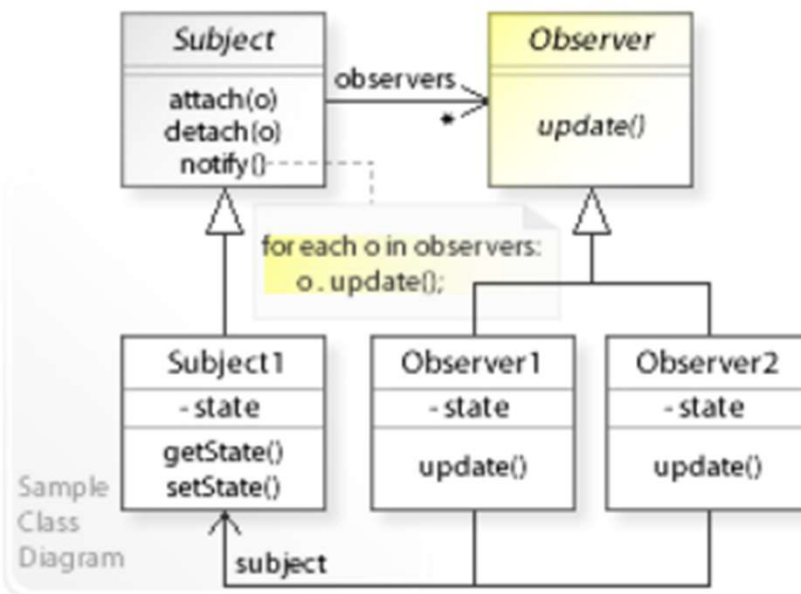
**To make subject and observers loosely coupled:**

- The sole responsibility of a subject is to maintain a list of observers and to notify them of state changes by calling their update() operation.
- The responsibility of observers is to register (and unregister) themselves on a subject (to get notified of state changes) and to update their state (synchronize their state with subject's state) when they are notified.

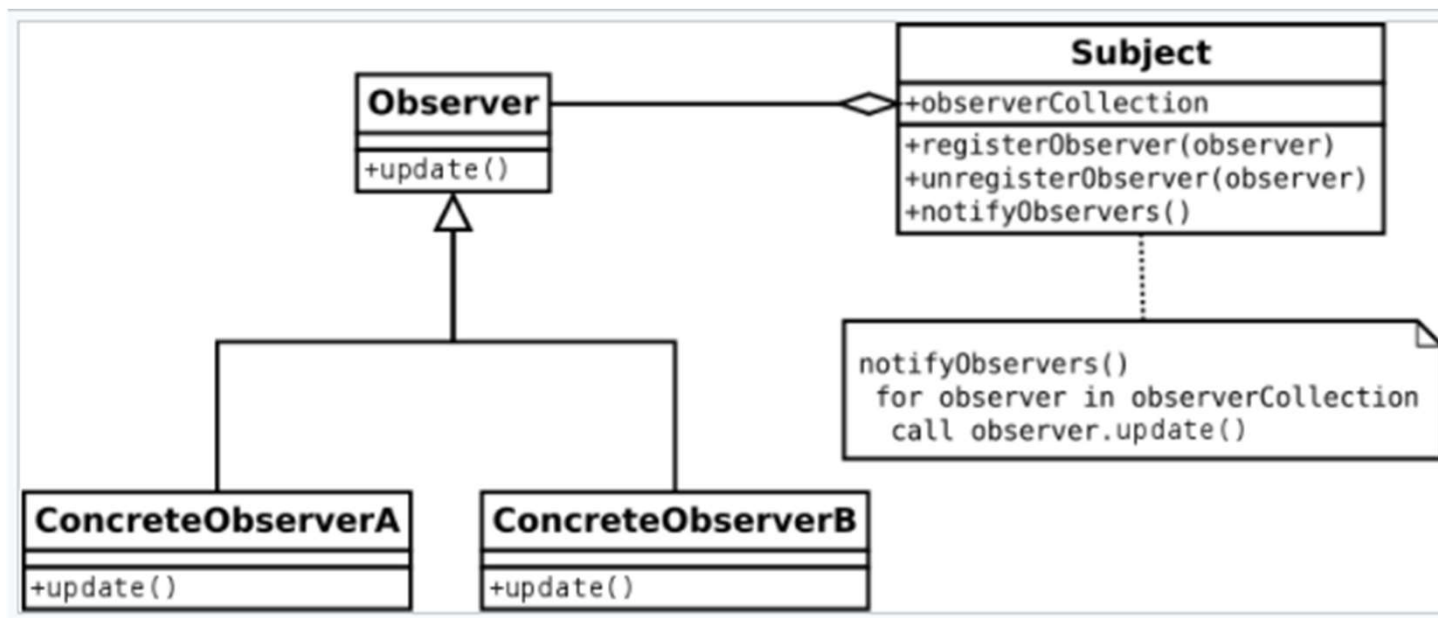
**Note:**

- Subject and observers have no explicit knowledge of each other.
- Observers can be added and removed independently at run-time.
- This notification-registration interaction is also known as publish-subscribe.

## Class diagram and sequence diagram



## Another class diagram



**Question 1a:** Identify the design problem in this feature and suggest a design pattern for addressing this problem.

**Design problems:**

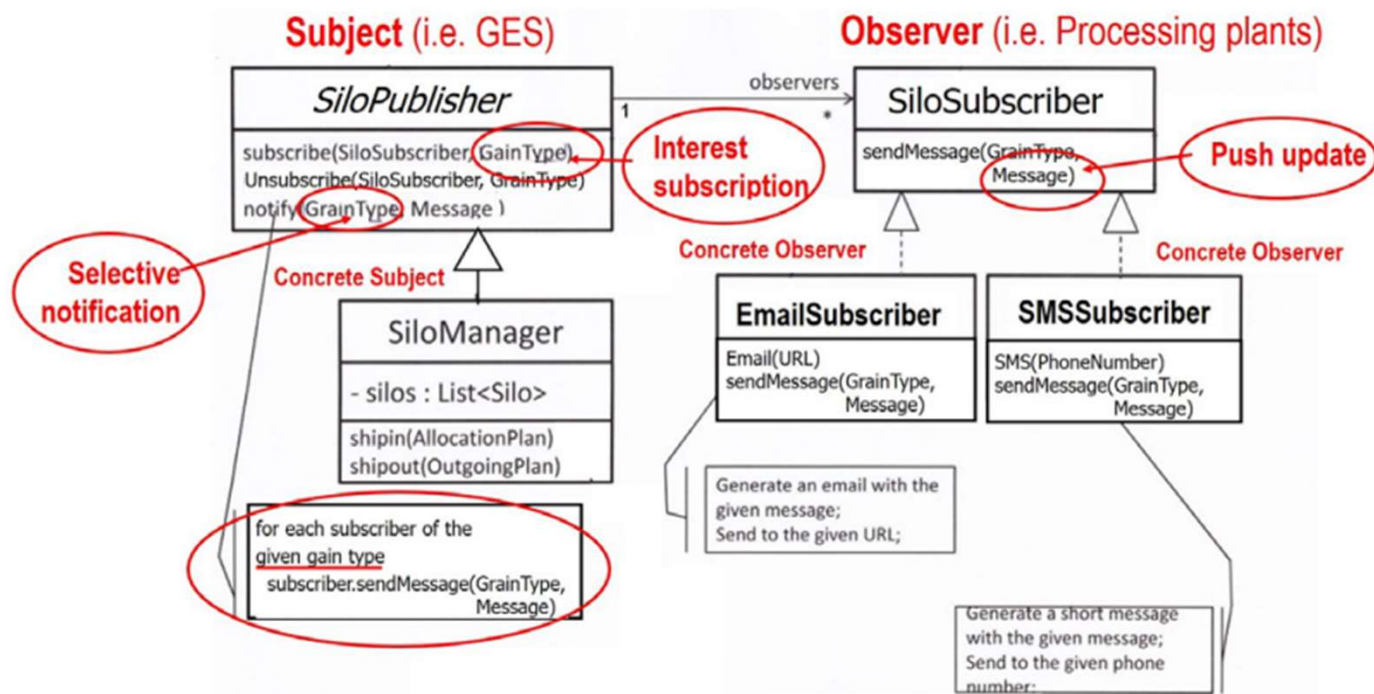
There is tight coupling between an object (subject) and its dependent objects (observers):

There should be **Loose coupling** and **immediate update** between subject and its dependent observers:

- Subject wants to notify observers the change once it occurs, but observers can change their interests freely and constantly - loose coupling.
- Observers want to know the change once it occurs, but they do not know when the change will occur – immediate update.

**Solution:** Observer pattern

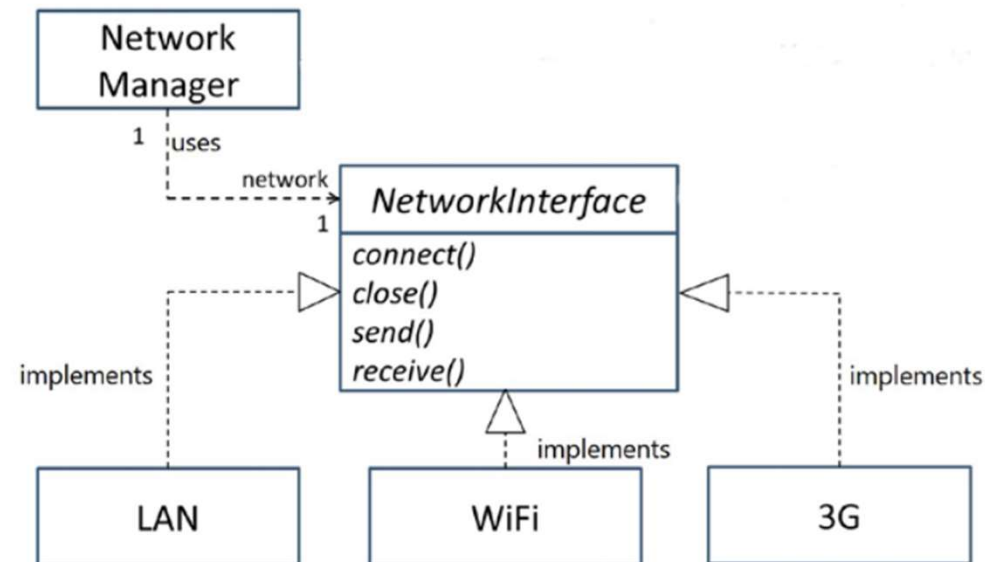
Question 1b: One possible class diagram.



|                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SiloPublisher<br>(Subject)        | <ul style="list-style-type: none"> <li>• This is the Subject abstract class (SiloPublisher) that defines operations to subscribe, unsubscribe, and notify the observers if there's any changes in the Subject. The <b>notify()</b> method (a selective notification) is to update all the silo subscribers (concrete observers) of their given interests (types of grains) as parameter via the <b>notify (GrainType, Message)</b> method, which calls the <b>sendMessage(GrainType, Message)</b> defines in the ConcreteObserver classes (EmailSubscriber, SMSSubscriber).</li> <li>• ConcreteObserver objects use the Subject (SiloPublisher) to subscribe as observers and also to remove themselves (unsubscribe) from being observers.</li> </ul> |
| SiloManager<br>(Concrete Subject) | <ul style="list-style-type: none"> <li>• This is the ConcreteSubject class (SiloManager), which is the object that is being observed.</li> <li>• It stores or maintains a list of all registered observers.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

|                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|--------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SiloSubscriber<br>(Observer)                                 | <ul style="list-style-type: none"> <li>This is the Observer Interface class that defines an <b>update()</b> method, i.e. <b>sendMessage(GrainType, Message)</b> for concrete observers to be notified of any changes (updates) in the Subject.</li> </ul>                                                                                                                                                                                                                                                                                                                                |
| EmailSubscriber,<br>SMSSubscriber<br>(Concrete<br>Observers) | <ul style="list-style-type: none"> <li>They are the ConcreteObserver classes that receive updates in their preferred communication means: Email, SMS.</li> <li>All concrete observers (EmailSubscriber, SMSSubscriber) implement the same Observer interface (SiloSubscriber).</li> <li>The concrete observers (EmailSubscriber, SMSSubscriber) can register their <b>interests</b> in specific types of grains by specifying the types of grains as a parameter in the subscribe method as <b>subscribe(SiloSubscriber, GrainType)</b> in the Subject class (SiloPublisher).</li> </ul> |



**Question 2: (refer to Question 2 of Tutorial 6)**

NetworkManager: Context

NetworkInterface: Strategy interface

LAN, WiFi, 3G, etc.: Concrete strategy objects

**Answer 2a: Missing in the solution:**

**How to handle object creation of different network interface objects.**

**Answer 2b: Combine Factory pattern with Strategy pattern.**

LocationConfigurator is the Factory.

